INTERNSHIP REPORT

PFE - Final Year Project

SPRINT 5

PHP Mastery : From Fundamentals to Frameworks

Full Name : Ayoub Barahal

Duration : 1 Week

Status : Done

"From syntax mastery to professional tooling and package development."

Table des matières

1 Introduction **2**

2 Chapter 1 : PHP Syntax, Setup & Tooling **2**
2.1 Step 1 : Educational Videos . . . 2
2.2 Step 2 : Hands-on Coding Practice . . . 2
2.2.1 Repository Setup . . . 2
2.2.2 Coding The PHP League Config Package . . . 2
2.3 Step 3 : Tooling Configuration . . . 3
2.4 Step 4 : Publishing a Dummy Package to Packagist . . . 3
2.4.1 composer.json Configuration . . . 3
2.4.2 download the package . . . 4
2.5 Chapter Summary . . . 4
2.5.1 Key Achievements . . . 4
2.5.2 Muscle Memory Training Results . . . 4

3 Chapter 2 : Build a Simple PHP MVC Framework **5**
3.1 Overview . . . 5
3.2 Project Repository . . . 5
3.3 Project Demonstration . . . 5
3.4 Chapter Summary . . . 5
3.4.1 Concepts Reinforced . . . 5

4 Chapter 3 : Working with Databases CLI **6**
4.1 Step 1 : Watch (Done) . . . 6
4.2 Step 2 : Hands-on Exercises . . . 6
4.2.1 Overview . . . 6
4.2.2 Symfony Console CLI Setup . . . 6
4.2.3 MySQL Database Setup : Create Database, User and Privileges . . . 6
4.2.4 ImportCsvCommand — PDO, Raw SQL & Transactions . . . 7
4.2.5 Export & Transfer Data . . . 8

5 Chapter 4 : REST API Using Laravel **9**
5.1 Objective . . . 9
5.2 Création du projet . . . 9
5.3 Configuration de l'environnement . . . 9
5.4 Base de données et migrations . . . 10
5.4.1 Création de la migration . . . 10
5.4.2 Exécution des migrations et du Seeder . . . 10
5.5 Middlewares personnalisés . . . 10
5.6 Endpoint API — Filtres, Tri et Pagination . . . 10
5.7 Tests using Postman . . . 10
5.7.1 Requête de base . . . 10
5.7.2 Requête avec filtres, tri et pagination . . . 11

6 Conclusion **14**

1 Introduction

Sprint 5 marks a significant shift from debugging to deep learning. This sprint focuses on mastering PHP fundamentals, understanding the modern PHP ecosystem, and building professional development practices.

Prerequisites

Before starting this training, I ensured I had :

- Read PHP: The Right Way
- Watched the brief overview : Loom Video
- Watched a professional coding session : PHP Coding Session

Objective

By the end of this training, develop an intermediate-level understanding of PHP and its ecosystem.

2 Chapter 1 : PHP Syntax, Setup & Tooling

Duration : 1 Day

2.1 Step 1 : Educational Videos

2.2 Step 2 : Hands-on Coding Practice

2.2.1 Repository Setup

```
# Create new Git repository
mkdir php-syntax-lab
cd php-syntax-lab
git init
git add README.md
git commit -m "Initial commit: Sprint 5 repository"
```

Listing 1 – Git repository initialization

2.2.2 Coding The PHP League Config Package

GitHub Repository

https://github.com/abarahal/my-php-framework

2.3 Step 3 : Tooling Configuration

2.4 Step 4 : Publishing a Dummy Package to Packagist

2.4.1 composer.json Configuration

Users > void > php-syntax-lab > {} composer.json > ...

```
{
  "name": "ayoubbarahal/dummy-package",
  "description": "This package should not be used; it is for testing purposes only.",
  "type": "library",
  "require": {
    "php": ">=8.0"
  },
  "autoload": {
    "psr-4": {
      "DummyPackage\\": "src/"
    }
  },
  "license": "MIT",
  "authors": [
    {
      "name": "ayoubbarahal",
      "email": "a.barahal@void.fr"
    }
  ]
}
```

FIGURE 1 – composer.json

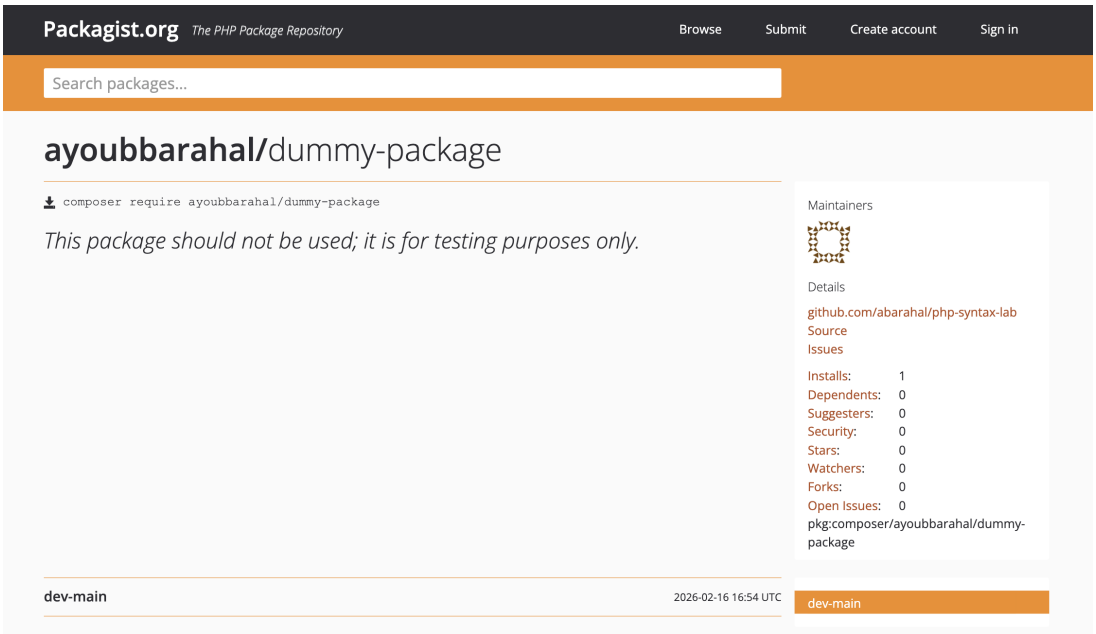

FIGURE 2 – submit the package

2.4.2 download the package

```
test — void@VOIDs-MacBook-Pro — -zsh — 80×24
..hp-syntax-lab                                  ~/test
→  test composer require ayoubbarahal/dummy-package:dev-main

Composer could not detect the root package (ayoubbarahal/test) version, defaulti
ng to '1.0.0'. See https://getcomposer.org/root-version
./composer.json has been updated
Composer could not detect the root package (ayoubbarahal/test) version, defaulti
ng to '1.0.0'. See https://getcomposer.org/root-version
Running composer update ayoubbarahal/dummy-package
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking ayoubbarahal/dummy-package (dev-main c9c9d82)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading ayoubbarahal/dummy-package (dev-main c9c9d82)
  - Installing ayoubbarahal/dummy-package (dev-main c9c9d82): Extracting archive
Generating autoload files
No security vulnerability advisories found.
→  test ls
composer.json composer.lock vendor
→  test
```

FIGURE 3 – download the package

2.5 Chapter Summary

2.5.1 Key Achievements

- **Syntax Mastery :** Typed complete configuration package manually, reinforcing PHP syntax patterns
- **Tooling Setup :** Configured PHPCS and PHPStan for professional development workflow
- **Package Publishing :** Successfully published a test package to Packagist
- **Educational Foundation :** Watched comprehensive video resources and began structured learning

2.5.2 Muscle Memory Training Results

- Comfortable with PHP variable syntax ($variable)
- Object operator (->) becomes natural
- Type declarations and strict typing
- Namespace and use statements pattern
- Array syntax and operations
- Function and method definitions

3 Chapter 2 : Build a Simple PHP MVC Framework

Duration : 1 Day

3.1 Overview

This chapter focuses on building a lightweight PHP MVC (Model-View-Controller) framework from scratch. Rather than relying on an existing framework, the goal is to deeply understand the internals of how modern PHP frameworks operate — from routing and request handling to template rendering — by constructing each component manually.

3.2 Project Repository

The tutorial "Write Your Own PHP MVC Framework" was followed step by step .
The complete source code of the custom PHP MVC framework built during this chapter is available on GitHub :

GitHub Repository

https://github.com/abarahal/my-php-framework

3.3 Project Demonstration

This video showcases my To-Do List project, demonstrating how my PHP framework manages and displays different actions like adding, deleting, and consulting tasks :

Demo Video

https://www.loom.com/share/0f5361fcd41c4d6182ac646d1edb6769

3.4 Chapter Summary

3.4.1 Concepts Reinforced

- Understanding of how Laravel, Symfony, and other frameworks handle requests internally
- PHP OOP principles : inheritance, abstract classes, interfaces
- Apache server configuration and virtual host setup
- Clean URL design and RESTful routing conventions

4 Chapter 3 : Working with Databases CLI

4.1 Step 1 : Watch (Done)

4.2 Step 2 : Hands-on Exercises

4.2.1 Overview

The goal of this exercise was to build a CLI application capable of importing a CSV file into a MySQL database using Symfony's Console Component and PHP PDO.

4.2.2 Symfony Console CLI Setup

The CLI application was built using the **Symfony Console Component**, which allows creating structured and professional command-line commands.

GitHub Repository

https://github.com/abarahal/my-php-framework

4.2.3 MySQL Database Setup : Create Database, User and Privileges

```
mysql> create user 'csv_user'@'localhost' identified by 'Csv@1234'
    -> ;
Query OK, 0 rows affected (0.03 sec)

mysql>
```

FIGURE 4 – create user

```
mysql> CREATE DATABASE csv_import_db;
Query OK, 1 row affected (0.02 sec)

mysql> GRANT ALL PRIVILEGES ON csv_import_db.* TO 'csv_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql>
```

FIGURE 5 – create database and grant privilege

```
mysql> CREATE TABLE customers (
    ->     id                INT PRIMARY KEY,
    ->     customer_id       VARCHAR(50),
    ->     first_name        VARCHAR(100),
    ->     last_name         VARCHAR(100),
    ->     company           VARCHAR(150),
    ->     city              VARCHAR(100),
    ->     country           VARCHAR(100),
    ->     phone1            VARCHAR(50),
    ->     phone2            VARCHAR(50),
    ->     email             VARCHAR(150),
    ->     subscription_date DATE,
    ->     website           VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
```

FIGURE 6 – create customers table

4.2.4 ImportCsvCommand — PDO, Raw SQL & Transactions

The `ImportCsvCommand` class handles the full import pipeline : reading the CSV, inserting rows using **PHP PDO** with **raw SQL**, wrapped in a **database transaction** to ensure data integrity. The script execution time is also measured.

```
→  DataMigrator php bin/console.php app:import-csv
Import réussi !
Import terminé en 0.99 secondes.
→  DataMigrator
```

FIGURE 7 – stored successfully

```
mysql> select count(*) from customers ;
+----------+
| count(*) |
+----------+
|    10000 |
+----------+
1 row in set (0.01 sec)

mysql>
```

FIGURE 8 – data added in the table

4.2.5 Export & Transfer Data

Export with mysqldump Once the import was complete, the database was exported into a `.sql` file using `mysqldump`, then reimported to verify integrity.

```
→  DataMigrator mysqldump -u csv_user -p csv_import_db > export.sql

Enter password:
Warning: A partial dump from a server that has GTIDs will by default incl
 GTIDs of all transactions, even those that changed suppressed parts of t
base. If you don't want to restore GTIDs, pass --set-gtid-purged=OFF. To
complete dump, pass --all-databases --triggers --routines --events.
Warning: A dump from a server that has GTIDs enabled will by default incl
 GTIDs of all transactions, even those that were executed during its extr
and might not be represented in the dumped data. This might result in an
stent data dump.
In order to ensure a consistent backup of the database, pass --single-tra
n or --lock-all-tables or --source-data.
→  DataMigrator ls
bin           composer.lock export.sql    vendor
composer.json data          src
→  DataMigrator
```

FIGURE 9 – export data

```
→  DataMigrator mysql -u csv_user -p csv_import_db < export.sql

Enter password:
→  DataMigrator
```

FIGURE 10 – import data

5 Chapter 4 : REST API Using Laravel

5.1 Objective

The objective of this chapter is to design and implement a secure REST API using Laravel, including filtering, sorting, pagination, and authentication features.

5.2 Création du projet

La création du projet laravel via Composer :

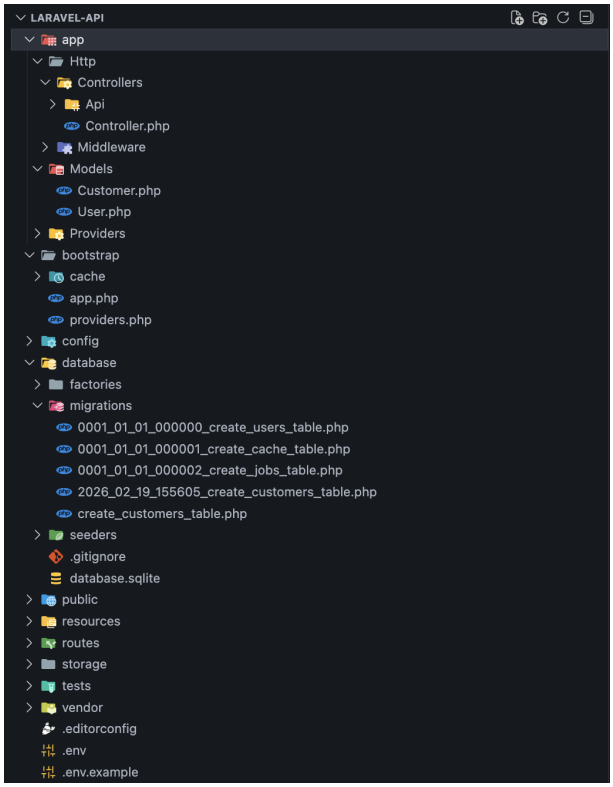

FIGURE 11 – Structure de Projet Laravel

5.3 Configuration de l'environnement

Le fichier `.env` a été configuré pour établir la connexion avec la base de données MySQL :

```
22
23  DB_CONNECTION=mysql
24  DB_HOST=127.0.0.1
25  DB_PORT=3306
26  DB_DATABASE=customers
27  DB_USERNAME=root
28  DB_PASSWORD=Ayoub@2004
29
```

FIGURE 12 – configuration de .env

5.4 Base de données et migrations

5.4.1 Création de la migration

5.4.2 Exécution des migrations et du Seeder

```
php artisan migrate
php artisan db:seed --class=DatabaseSeeder
```

Listing 2 – Migration et alimentation de la base

5.5 Middlewares personnalisés

```
3 references | 0 implementations
class BasicAuthMiddleware
{
    1 reference
    private const VALID_USER     = 'admin';
    1 reference
    private const VALID_PASSWORD = 'secret';

    0 references | 0 overrides
    public function handle(Request $request, Closure $next): mixed
    {
        $user     = $request->getUser();
        $password = $request->getPassword();

        if ($user !== self::VALID_USER || $password !== self::VALID_PASSWORD) {
            return response()->json(
                data: ['error' => 'Unauthorized'],
                status: 401,
                headers: ['WWW-Authenticate' => 'Basic realm="API"']
            );
        }

        return $next($request);
    }
}
```

FIGURE 13 – configuration de .env

5.6 Endpoint API — Filtres, Tri et Pagination

UserController

https://github.com/abarahal/my-php-framework

5.7 Tests using Postman

5.7.1 Requête de base

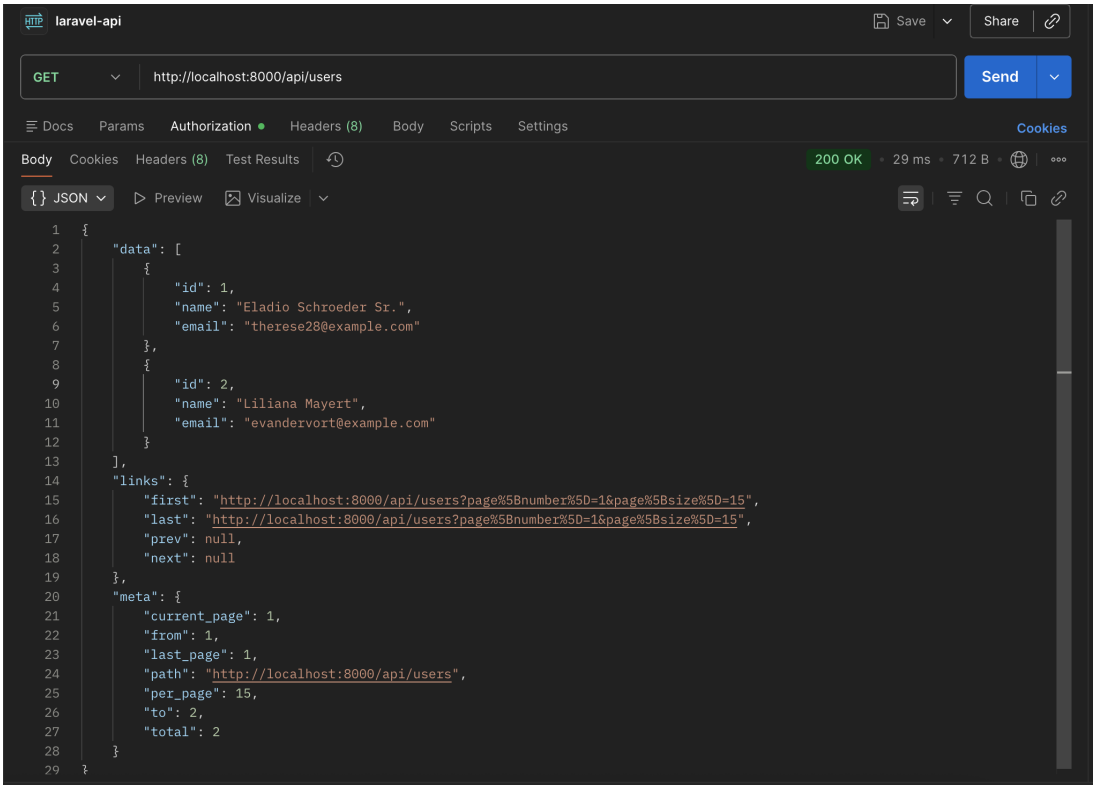

FIGURE 14 – basic test using postman

5.7.2 Requête avec filtres, tri et pagination

```
➜  ~ curl -g -u admin:secret \
  'http://localhost:8000/api/users' \
  -H 'Accept: application/json' | python3 -m json.tool
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   472    0   472    0     0  18304      0 --:--:-- --:--:-- --:--:-- 18880
{
    "data": [
        {
            "id": 1,
            "name": "Eladio Schroeder Sr.",
            "email": "therese28@example.com"
        },
        {
            "id": 2,
            "name": "Liliana Mayert",
            "email": "evandervort@example.com"
        }
    ],
    "links": {
        "first": "http://localhost:8000/api/users?page%5Bnumber%5D=1&page%5Bsize%5D=15",
        "last": "http://localhost:8000/api/users?page%5Bnumber%5D=1&page%5Bsize%5D=15",
        "prev": null,
        "next": null
    },
    "meta": {
        "current_page": 1,
        "from": 1,
        "last_page": 1,
        "path": "http://localhost:8000/api/users",
        "per_page": 15,
        "to": 2,
        "total": 2
    }
```

FIGURE 15 – basic test using curl

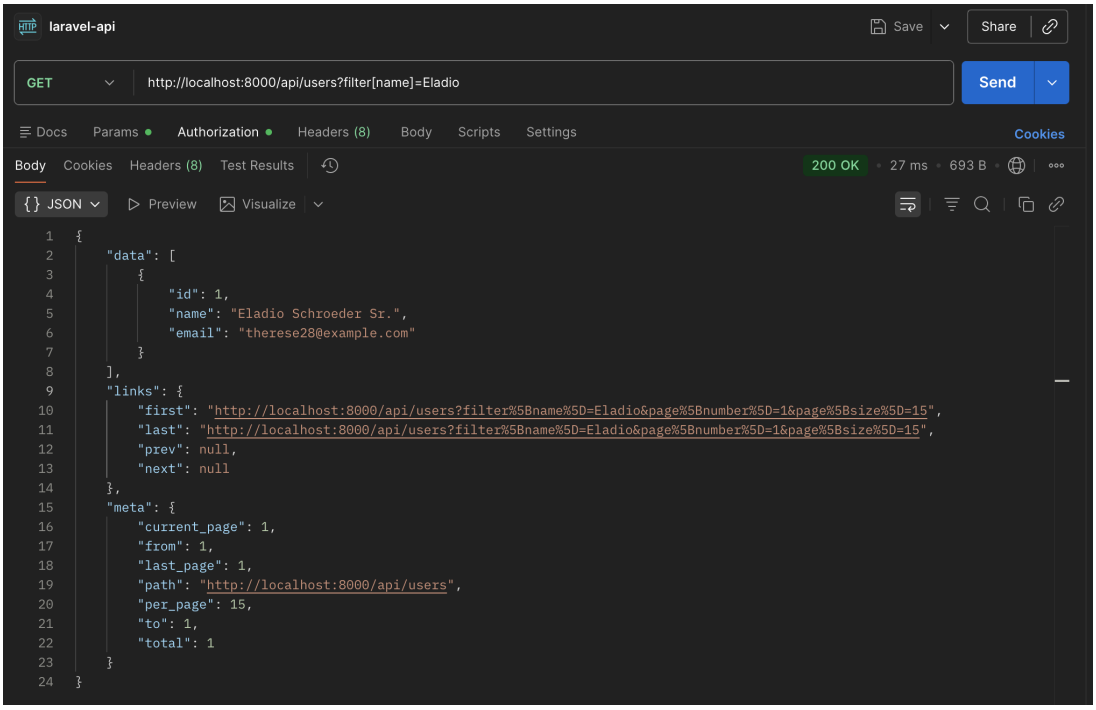

FIGURE 16 – filter the name

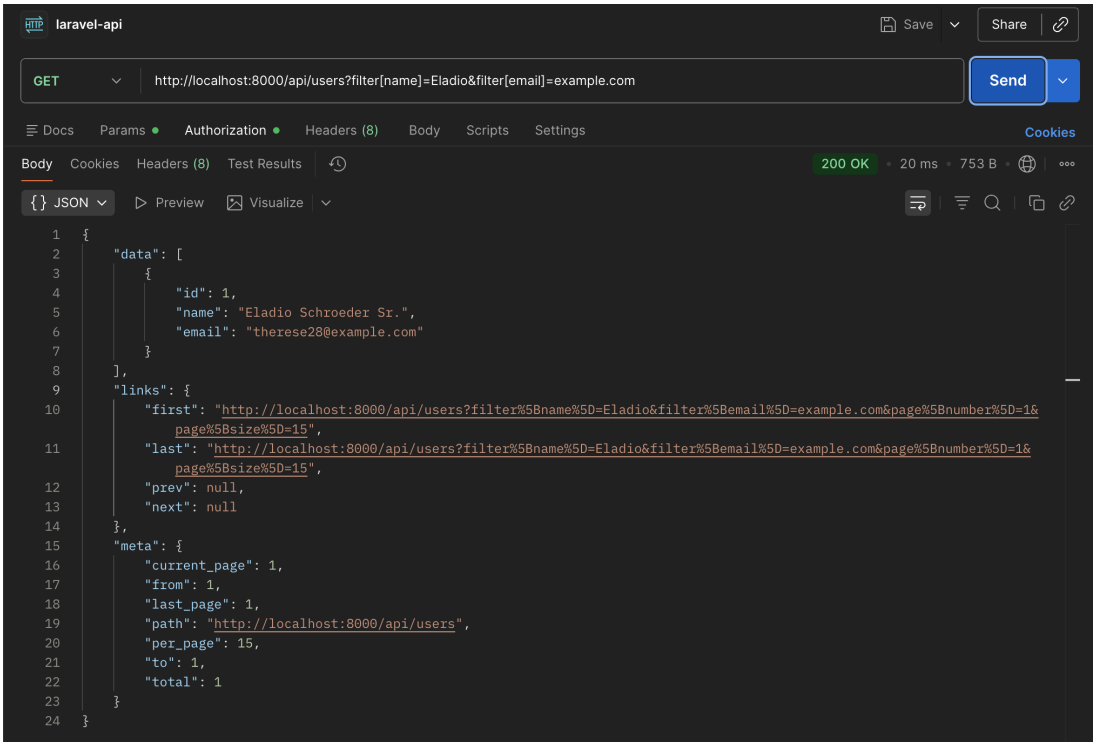

FIGURE 17 – use the filtre , sort and pagination

FIGURE 18 – use the filtre , sort and pagination

6 Conclusion

Ce sprint a été une immersion complète dans l'écosystème PHP moderne. En partant des fondamentaux du langage jusqu'au développement d'une API REST avec Laravel, j'ai progressivement construit une vision globale du développement PHP professionnel.

Les exercices pratiques — retranscription manuelle de code, création de frameworks from scratch, import CSV via CLI, et conception d'une API RESTful — m'ont permis de consolider des compétences concrètes et directement applicables en environnement de production.

La configuration des outils de qualité (PHPCS, PHPStan) et la publication d'un package sur Packagist m'ont mis en évidence l'importance des bonnes pratiques et des standards dans le développement PHP moderne.

Enfin, ce sprint m'a démontré que PHP, loin de ses clichés, est un langage mature, performant et doté d'un écosystème riche, capable de répondre aux exigences des applications web modernes.